
Screening Is Enough

Ken M. Nakanishi

Center for Emergent Matter Science (CEMS), RIKEN
 Graduate School of Science, The University of Tokyo
 ken.m.nakanishi@gmail.com

Abstract

A core limitation of standard softmax attention is that it does not provide an independently interpretable measure of query–key relevance: attention scores are unbounded, while attention weights are defined only relative to competing keys. Consequently, irrelevant keys cannot be explicitly rejected, and some attention mass is assigned even when no key is genuinely relevant. We introduce Multiscreen, a language-model architecture built around a mechanism we call screening, which enables absolute query–key relevance. Instead of redistributing attention across all keys, screening computes bounded query–key similarities and applies an explicit threshold, discarding irrelevant keys and aggregating the remaining keys without global competition. Across experiments, Multiscreen achieves comparable validation loss with roughly 30% fewer parameters than a Transformer baseline and remains stable at substantially larger learning rates. It maintains stable long-context perplexity beyond the training context and shows little degradation in retrieval performance as context length increases. Finally, Multiscreen achieves lower full-context forward-pass latency at long context lengths.

1 Introduction

Handling contexts substantially longer than those seen during training remains a central challenge for large language models (LLMs). Long contexts are crucial for using in-context information effectively, but training on long sequences is computationally expensive because the cost of Transformer self-attention grows quadratically with sequence length [1]. As a result, models are often trained on relatively short contexts and then expected to generalize to much longer ones at inference time. However, simply increasing the nominal context length does not guarantee that a model can effectively use relevant information within it.

A key difficulty lies in how relevant information is selected from the context. Standard softmax attention does not provide an independently interpretable measure of query–key relevance. Attention scores are unbounded and scale-dependent, making their magnitudes difficult to interpret in isolation, while attention weights are defined only after redistributing a fixed unit mass across all keys. Thus, the apparent relevance of a key depends on competing keys rather than on the query–key pair alone. Moreover, irrelevant keys cannot be cleanly rejected without masking, and even when no key is genuinely relevant, some attention mass must still be assigned. As contexts grow longer, this global competition can dilute attention over many keys, making it increasingly difficult to preserve strong signals from relevant tokens.

We introduce Multiscreen, a language-model architecture built around a mechanism we call screening, which enables absolute query–key relevance. Instead of redistributing attention across all keys as in softmax attention, screening computes bounded query–key similarities and transforms them into relevance values through an explicit threshold. Irrelevant keys can be assigned exactly zero relevance, while the remaining keys are aggregated without competition among keys. This allows the model to represent the absence of relevant context.

Multiscreen organizes screening into parallel gated screening tiles. Each tile learns a screening window that controls its effective context range, allowing the model to avoid unnecessary long-range computation. Multiscreen also uses minimal positional encoding (MiPE), which rotates only two dimensions and is active only for small screening windows, becoming inactive when the window exceeds a fixed threshold. Thus, large-window tiles do not rely on extrapolating positional rotations beyond those seen during training.

Validation loss alone does not reveal whether a model can retrieve and use relevant information from long contexts. To directly evaluate retrieval, we introduce **ABCDigits**, a semantics-free key-value retrieval benchmark designed to isolate retrieval behavior from natural-language semantics and instruction-following effects.

Empirically, Multiscreen improves several aspects of language modeling at once. Across scaling experiments, it achieves similar validation loss with roughly 30% fewer parameters than Transformer baselines under the same token budget. It remains stable at substantially larger learning rates, whereas Transformer training becomes unstable or diverges. On long-context perplexity, Multiscreen maintains stable performance beyond the training context length, while Transformer exhibits sharp degradation. On retrieval, Multiscreen shows little degradation as context length increases. Finally, Multiscreen achieves lower full-context forward-pass latency in long-context settings.

These results indicate that improving long-context behavior requires moving beyond redistribution-based mechanisms toward architectures that select information using absolute relevance.

Our main contributions are as follows:

- We introduce **Multiscreen**, a language-model architecture based on *screening*, which enables absolute query-key relevance.
- We show that Multiscreen improves parameter efficiency, optimization stability, long-context perplexity, retrieval ability, and inference latency compared to Transformer baselines.
- We introduce **ABCDigits**, a semantics-free completion-based retrieval benchmark designed to isolate retrieval behavior without relying on natural-language semantics or instruction-following.

2 Related Work

Attention mechanisms and relevance. A large body of work modifies softmax attention to improve sparsity, length generalization, or efficiency, while retaining normalized weighting across keys. Scalable-Softmax sharpens attention as context length grows [2], Selective Attention introduces query- and position-dependent temperature scaling [3], and Forgetting Attention adds a data-dependent forget gate before softmax [4]. Other work replaces softmax with sparse normalized mappings such as sparsemax or entmax [5, 6, 7], or restricts the set of attended keys through sparse or retrieval-based mechanisms [8, 9, 10, 11]. These approaches improve attention behavior or efficiency, but still rely on normalized weighting across keys.

A more direct departure from softmax is sigmoid attention, which weights query-key pairs independently and removes competition across keys [12]. However, sigmoid attention assigns strictly positive weights and cannot exactly reject irrelevant keys, and uses sequence-length-dependent bias terms to control the growth of attention magnitude. Multiscreen differs from both softmax-based and sigmoid-based approaches by defining bounded query-key similarities and applying an explicit threshold-and-rescale transform, enabling exact rejection and independence across keys. This yields relevance values that are independent across keys, can be exactly zero, and do not require sequence-length-dependent calibration.

Beyond attention, alternative sequence modeling approaches replace explicit token-to-token interaction with mechanisms such as state-space models, long convolutions, or retention [13, 14, 15]. Such architectures improve efficiency, but recall and associative retrieval can remain challenging [16]; Multiscreen instead preserves explicit query-key comparisons while reducing unnecessary computation through learned screening windows.

Long-context positional representations. Length generalization has also been studied through positional representations. Representative approaches include ALiBi and RoPE [17, 18], RoPE

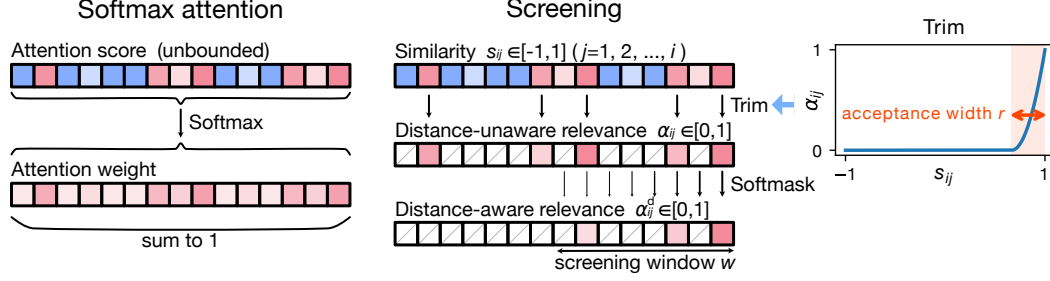


Figure 1: Conceptual comparison between softmax attention and screening. Each cell corresponds to a scalar value associated with an individual key. Colors indicate scalar values (blue: negative, red: positive; intensity indicates magnitude), while hatched cells denote zero values. Softmax attention maps unbounded attention scores to nonnegative attention weights that sum to one, redistributing mass across all keys. In contrast, screening maps bounded query–key similarities to distance-unaware relevance via Trim, which assigns exactly zero relevance to similarities below a threshold, and then applies Softmask to produce distance-aware relevance.

interpolation and scaling methods such as Position Interpolation, YaRN, and LongRoPE [19, 20, 21], and learned or functional relative position schemes such as FIRE [22]. Other work studies removing explicit positional encodings altogether, including NoPE and analyses of length generalization without position encodings [23, 24]. Multiscreen uses minimal positional encoding (MiPE), which rotates only two dimensions and is applied only for small screening windows; the rotation angle smoothly decays to zero as the screening window approaches a fixed threshold. Thus, behavior in large-window tiles is governed primarily by learned screening windows rather than extrapolated positional patterns.

Retrieval evaluation and synthetic benchmarks. Long-context evaluation has shown that models may fail to use relevant information even when it lies within the nominal context window, as in lost-in-the-middle phenomena [25]. Synthetic recall and key–value retrieval tasks are commonly used to probe memory and retrieval behavior [26, 27, 16], with needle-in-a-haystack and passkey-style benchmarks targeting long-context retrieval [28, 29]. At the same time, retrieval evaluations can be affected by semantic cues, semantic masking, or prompt-specific effects [30]. ABCDigits isolates retrieval in a semantics-free completion setting with a fixed key set and a uniquely determined target value, complementing existing retrieval-oriented benchmarks.

3 Model Architecture

In softmax attention, query–key dot products define unbounded attention scores, while attention weights are obtained by redistributing a fixed unit mass across all keys. Thus, neither the scores nor the weights provide an independently interpretable measure of query–key relevance, making it difficult to explicitly reject irrelevant keys or represent the absence of relevant context.

Multiscreen addresses this limitation with a mechanism called *screening*, which enables absolute query–key relevance. In screening, unit-normalized query–key dot products define bounded similarities in $[-1, 1]$. These similarities are transformed via Trim and Softmask into relevance values, which are then used directly for value aggregation. This contrast between softmax-based relative weighting and screening-based relevance assignment is illustrated in fig. 1.

The overall architecture is illustrated in fig. 2a. Each layer contains parallel gated screening tiles (see fig. 2b), which project token representations into query, key, value, and gate vectors, apply a screening unit to retrieve relevant context, and gate the result using a GLU-style multiplicative mechanism [31, 32, 33] before projecting back to the embedding dimension. In the actual implementation, several operations are fused, and terms outside the learned screening window are skipped for efficiency.

3.1 Overview

The overall architecture is illustrated in fig. 2a. Let $W_E = [e_1; \dots; e_{|V|}]$ denote the embedding matrix, and define $\bar{e}_j = e_j / \|e_j\|$ as the unit-length normalization of e_j . Given a token t_i , its input

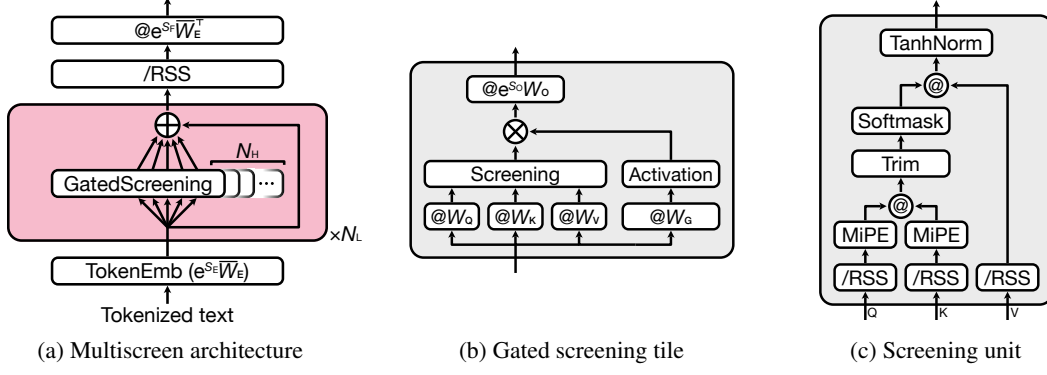


Figure 2: (a) Multiscreen consists of N_L residual layers, each containing N_H parallel gated screening tiles. The input embedding matrix is row-wise unit-normalized and shared with the language-modeling head, with learned scalars e^{s_E} and e^{s_F} controlling scaling. $\bar{W}_E$ denotes the row-wise unit-normalized embedding matrix. (b) A gated screening tile computes query, key, value, and gate projections; applies a screening unit; modulates the result with an activation gate; and projects back to the embedding dimension. (c) A screening unit normalizes queries, keys, and values; applies minimal positional encoding (MiPE); computes relevance through Trim and Softmask; aggregates surviving values; and applies TanhNorm. In the diagrams, “@” denotes matrix multiplication and “/RSS” denotes row-wise unit-length normalization.

representation is

$$\mathbf{x}_i^{(0)} = e^{s_E} \bar{\mathbf{e}}_{t_i}, \quad (1)$$

where s_E is a learned scalar.

Multiscreen then applies N_L residual layers, each composed of N_H parallel gated screening tiles. Let $\Delta \mathbf{x}_i^{(\ell, h)}$ denote the update produced for position i by tile h in layer ℓ . The layer update is

$$\mathbf{x}_i^{(\ell)} = \mathbf{x}_i^{(\ell-1)} + \sum_{h=1}^{N_H} \Delta \mathbf{x}_i^{(\ell, h)}. \quad (2)$$

After the final layer, logits are computed using the same unit-normalized embeddings:

$$z_{ij} = \mathbf{x}_i^{(N_L)} (e^{s_F} \bar{\mathbf{e}}_j^T), \quad j \in \{1, \dots, |\mathcal{V}|\}, \quad (3)$$

where s_F is a learned scalar. A standard softmax over logits gives next-token probabilities.

3.2 Screening Unit

A screening unit takes projected query, key, and value vectors $\mathbf{q}_i, \mathbf{k}_i \in \mathbb{R}^{1 \times d_k}$ and $\mathbf{v}_i \in \mathbb{R}^{1 \times d_v}$, and returns a context-dependent representation $\mathbf{u}_i \in \mathbb{R}^{1 \times d_v}$ (see fig. 2c). Each unit has two learned scalar parameters, s_w and s_r , which define

$$w = e^{s_w} + 1, \quad r = \sigma(s_r), \quad (4)$$

where σ denotes the sigmoid function. Here w is the screening window and r is the acceptance width for similarity.

We first apply unit-length normalization to queries, keys, and values:

$$\bar{\mathbf{q}}_i = \frac{\mathbf{q}_i}{\|\mathbf{q}_i\|}, \quad \bar{\mathbf{k}}_i = \frac{\mathbf{k}_i}{\|\mathbf{k}_i\|}, \quad \bar{\mathbf{v}}_i = \frac{\mathbf{v}_i}{\|\mathbf{v}_i\|}. \quad (5)$$

In implementation, the denominator in each unit-length normalization is clipped from below by a small positive constant for numerical stability. Normalizing queries and keys bounds query–key similarities in $[-1, 1]$, making thresholding well-defined. Normalizing values prevents large value norms from dominating aggregation [34, 35].

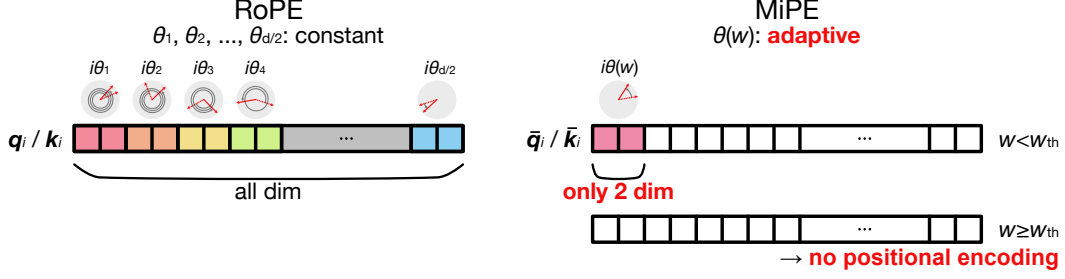


Figure 3: Comparison between RoPE and MiPE. RoPE applies rotations across all dimensions with fixed frequencies. MiPE applies rotation only to two dimensions and adapts the rotation angle based on the learned screening window w , becoming inactive when $w \geq w_{th}$.

Before computing similarity, we apply minimal positional encoding (MiPE; see fig. 3) to $\tilde{\mathbf{q}}_i$ and $\tilde{\mathbf{k}}_i$. MiPE is a RoPE-like rotation [18] applied only to the first two dimensions, with a rotation angle modulated by the learned screening window w . For position i , MiPE rotates by angle

$$\phi(i, w) = \frac{\pi i \gamma(w)}{w}, \quad (6)$$

where $\gamma(w)$ smoothly decreases to zero as w approaches w_{th} :

$$\gamma(w) = \begin{cases} \frac{1}{2} \left(\cos \frac{\pi w}{w_{th}} + 1 \right), & w < w_{th}, \\ 0, & w \geq w_{th}. \end{cases} \quad (7)$$

Thus MiPE is active only for small screening windows and becomes the identity when $w \geq w_{th}$.

Let $\tilde{\mathbf{q}}_i$ and $\tilde{\mathbf{k}}_j$ denote the resulting position-encoded query and key vectors. The bounded similarity is then

$$s_{ij} = \tilde{\mathbf{q}}_i \tilde{\mathbf{k}}_j^\top, \quad s_{ij} \in [-1, 1]. \quad (8)$$

We apply a Trim transform (see fig. 1) to obtain distance-unaware relevance:

$$\alpha_{ij} = \left[\max \left(1 - \frac{1 - s_{ij}}{r}, 0 \right) \right]^2. \quad (9)$$

This sets relevance exactly to zero whenever $s_{ij} \leq 1 - r$, and smoothly emphasizes similarities close to one.

We next apply a causal distance-aware softmax parameterized by the screening window w :

$$m_{ij}(w) = \begin{cases} \frac{1}{2} \left(\cos \frac{\pi(j-i)}{w} + 1 \right), & -w < j - i \leq 0, \\ 0, & \text{otherwise,} \end{cases} \quad (10)$$

and define distance-aware relevance

$$\alpha_{ij}^d = \alpha_{ij} m_{ij}(w). \quad (11)$$

A key contributes only if it survives both content-based and distance-based screening.

Finally, the screening unit aggregates the surviving values:

$$\mathbf{h}_i = \sum_{j \leq i} \alpha_{ij}^d \bar{\mathbf{v}}_j. \quad (12)$$

Because the relevance values $\{\alpha_{ij}^d\}_{j \leq i}$ are not constrained to sum to one, the unit can represent the absence of relevant context. To softly bound the output norm while preserving direction, we define the function

$$\text{TanhNorm}(\mathbf{x}) = \frac{\tanh \|\mathbf{x}\|}{\|\mathbf{x}\|} \mathbf{x}. \quad (13)$$

For sufficiently small $\|\mathbf{x}\|$, we use the limiting value $\text{TanhNorm}(\mathbf{x}) = \mathbf{x}$. The screening output is

$$\mathbf{u}_i = \text{TanhNorm}(\mathbf{h}_i). \quad (14)$$

Overall, screening defines relevance independently for each key, can assign exactly zero relevance to irrelevant keys, and aggregates only the surviving values.

3.3 Gated Screening Tile

A gated screening tile (see fig. 2b) combines screening-based context retrieval with input-dependent feature selection. Given token representations $\mathbf{x}_i \in \mathbb{R}^{1 \times d_E}$, the tile computes

$$\mathbf{q}_i = \mathbf{x}_i W_Q, \quad \mathbf{k}_i = \mathbf{x}_i W_K, \quad \mathbf{v}_i = \mathbf{x}_i W_V, \quad \mathbf{g}_i = \mathbf{x}_i W_G, \quad (15)$$

where $W_Q, W_K \in \mathbb{R}^{d_E \times d_K}$ and $W_V, W_G \in \mathbb{R}^{d_E \times d_V}$.

Using the screening unit from section 3.2, the tile retrieves

$$\mathbf{u}_i = \text{Screening}(\{\mathbf{q}_j, \mathbf{k}_j, \mathbf{v}_j\}_{j=1}^T)_i. \quad (16)$$

In parallel, it computes a gate via a nonlinear activation

$$\hat{\mathbf{g}}_i = \tanh(\text{SiLU}(\mathbf{g}_i)), \quad (17)$$

where SiLU is the sigmoid-weighted linear unit [36]. The tile output is

$$\Delta \mathbf{x}_i = (\mathbf{u}_i \odot \hat{\mathbf{g}}_i) (e^{s_O} W_O), \quad (18)$$

with $W_O \in \mathbb{R}^{d_V \times d_E}$ and learned scalar s_O .

This tile can be viewed as a generalization of GLU-style modules [31, 32, 33] and recent gated-attention variants [37], replacing the usual linear or attention-based transformation with screening-based context aggregation. Thus, each tile jointly performs context selection, feature gating, and projection back to the model space. Initialization details are provided in Appendix D. Architectural ablations of MiPE, TanhNorm, gating, and the learned acceptance width are provided in Appendix P.

4 Experiments

We compare Multiscreen with Transformer baselines across parameter efficiency, optimization stability, long-context performance, retrieval, and inference latency.

4.1 Experimental Setup

We compare Multiscreen with Transformer baselines on SlimPajama [38] under the same token budget.

Multiscreen uses AdamW [39] as in the Transformer baseline, but omits weight decay and gradient clipping, and is trained with a larger learning rate (2^{-4}), enabled by Multiscreen’s improved stability.

Full details, including model configurations and training hyperparameters, are provided in Appendix A.

4.2 Scaling Efficiency

We evaluate scaling behavior by comparing models trained with a fixed token budget. For smaller models, we report mean and standard deviation over three runs with different random seeds; larger models are evaluated with a single run.

As shown in fig. 4, Multiscreen is more parameter-efficient, achieving roughly 30% fewer parameters at similar validation loss along the scaling trend. Additional scaling analyses are provided in Appendix E.

4.3 Learning Rate Stability

We evaluate training stability across a learning-rate sweep, averaging three independent runs for each learning rate (see Appendix F for details).

As shown in fig. 5, Transformer becomes unstable and eventually diverges beyond moderate learning rates, whereas Multiscreen remains stable even at substantially larger values.

This improved stability enables the use of larger learning rates in practice. Additional analyses of training dynamics, including loss trajectories and gradient norms, are provided in Appendix G and Appendix H, offering further insight into the observed stability. We also evaluate a diagnostic Transformer baseline with QKNorm in Appendix I, which suggests that adding query-key normalization to Transformer does not reproduce the large-learning-rate stability of Multiscreen.

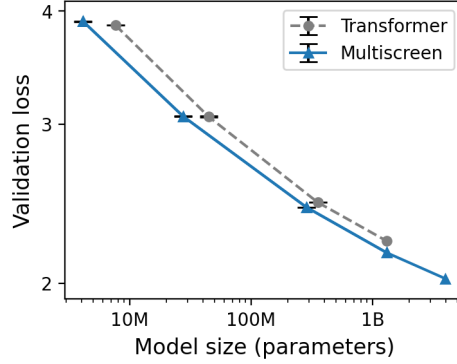


Figure 4: Validation loss vs. model size. Multiscreen achieves similar validation loss with roughly 30% fewer parameters along the scaling trend.

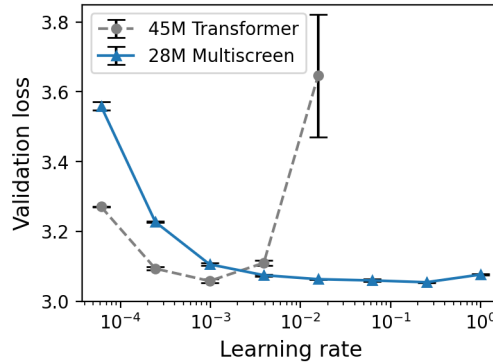


Figure 5: Learning rate sweep comparing Transformer and Multiscreen. Results are averaged over three independent runs for each learning rate, with error bars indicating standard deviation across runs. Multiscreen remains stable at large learning rates, while Transformer becomes unstable and diverges. For Transformer, runs with learning rates $\geq 2^{-4}$ diverged and are omitted.

4.4 Long-Context Perplexity

We evaluate long-context language modeling using position-dependent perplexity.

As shown in fig. 6, Multiscreen maintains stable perplexity as context length increases, without sharp degradation beyond the training context. In contrast, Transformer exhibits abrupt increases in perplexity once the context length exceeds the training range. Increasing the RoPE scaling factor delays breakdown but increases overall perplexity.

Additional evaluation details and seed-averaged smaller-model results are provided in Appendix L.

4.5 Retrieval Ability

To isolate retrieval ability, we introduce **ABCDigits**, a synthetic key–value retrieval benchmark in a semantics-free setting.

The task presents a shuffled list of mappings from uppercase letters to six-digit integers (e.g., A=967892), followed by a query (e.g., L=) requiring the model to retrieve the corresponding value. The target mapping appears exactly once in the context, requiring the model to locate a unique occurrence. The number of distinct keys is fixed, eliminating confounding effects from increasing key cardinality. Code for generating ABCDigits prompts is publicly available.¹

¹<https://github.com/ken-nakanishi/abcdigits>

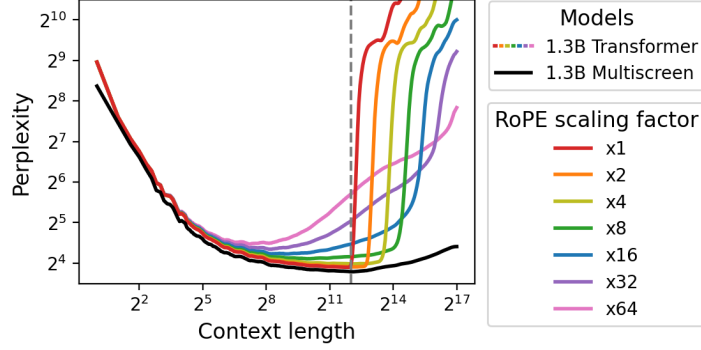


Figure 6: Long-context perplexity comparison. Multiscreen maintains stable perplexity beyond the training context, while Transformer degrades sharply. The dashed line marks the pretraining context length (2^{12}).

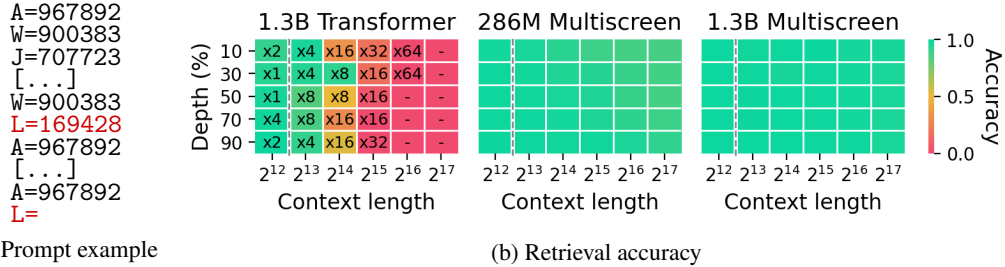


Figure 7: (a) Example ABCDigits prompt. (b) Retrieval accuracy over context length and target depth, averaged over 1,000 randomly generated instances per setting. For the 286M Multiscreen, results are further averaged across three independently trained models. Panels from left to right correspond to 1.3B Transformer, 286M Multiscreen, and 1.3B Multiscreen. The dashed line marks the pretraining context length (2^{12}). Multiscreen models show little degradation with increasing context length, while Transformer degrades substantially. For Transformer, each cell shows accuracy under the best-performing RoPE scaling factor selected from multiple candidates, with the selected factor indicated in each cell. A dash (“-”) indicates that no correct retrieval occurred.

We evaluate retrieval accuracy across context lengths from 2^{12} to 2^{17} and varying target depths (defined relative to the full context length), using exact-match accuracy under greedy decoding, averaged over 1,000 randomly generated instances per setting (and multiple models where applicable).

As shown in fig. 7, Multiscreen shows little degradation with increasing context length and maintains high retrieval accuracy beyond the training context length. Notably, even at the training context length, the 286M Multiscreen model outperforms the best RoPE-scaled 1.3B Transformer averaged over target depths (99.18% vs. 95.98%), with the gap becoming more pronounced at longer contexts. In contrast, Transformer performs substantially worse even with the best RoPE scaling selected from multiple candidates; its accuracy degrades beyond the training context and remains limited even within it.

These results highlight that standard language modeling metrics do not fully capture retrieval ability: despite having higher validation loss than the 1.3B Transformer, the 286M Multiscreen model achieves substantially stronger long-context retrieval. Additional ABCDigits details and scaling results, including optional inference-time screening-window expansion, are provided in Appendix M. Corresponding results on the standard passkey retrieval benchmark [29] are provided in Appendix N.

4.6 Inference Latency

We evaluate inference efficiency by measuring the latency of a single full-context forward pass for long input sequences. We consider two complementary settings: scaling across model sizes at a fixed context length of 2^{17} , and scaling across context lengths from 2^{10} to 2^{17} for 1.3B models.

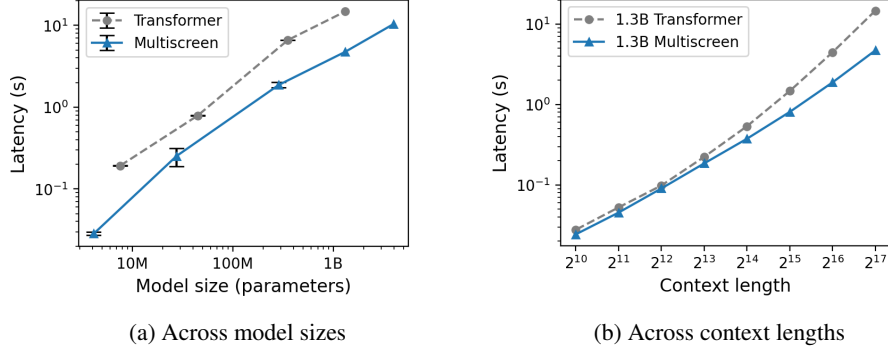


Figure 8: Inference latency for a single full-context forward pass. (a) Latency vs. model size at context length 2^{17} . Error bars indicate standard deviation across independently trained models, shown for smaller models. (b) Latency vs. context length for 1.3B Transformer and 1.3B Multiscreen models. Multiscreen achieves lower latency across model scales and context lengths, with the gap becoming more pronounced at longer contexts.

All measurements are conducted on an NVIDIA RTX 4090 GPU with batch size 1, causal masking, and mixed precision with bfloat16 matrix multiplications. We do not use KV caching; instead, the full input sequence is processed in a single forward pass. Transformer uses PyTorch scaled dot-product attention (`torch.nn.functional.scaled_dot_product_attention`) with the FlashAttention backend [40] explicitly enabled, while Multiscreen uses a custom Triton implementation [41] tailored to its screening mechanism.

For each trained model, we perform 100 repeated measurements and report the mean latency. In the model-size sweep, smaller models are averaged across three independently trained models, with error bars indicating standard deviation across models; larger models use a single trained model. In the context-length sweep, we evaluate one 1.3B Transformer and one 1.3B Multiscreen model.

As shown in fig. 8, Multiscreen achieves consistently lower latency than Transformer across model sizes. The context-length sweep further shows that this advantage persists across input lengths and becomes more pronounced as the context length increases.

5 Conclusion

In this work, we identified a fundamental limitation of standard softmax attention: it does not provide an independently interpretable measure of query-key relevance. Attention scores are unbounded, while attention weights are defined only relative to competing keys by redistributing a fixed unit mass across the context. Consequently, irrelevant keys cannot be explicitly rejected, and the model cannot directly represent the absence of relevant context through attention.

To address this limitation, we introduced Multiscreen, a language-model architecture built around a mechanism we call *screening*, which enables absolute query-key relevance. Instead of redistributing attention across all keys, screening computes bounded query-key similarities and transforms them into relevance values through an explicit threshold. This allows irrelevant keys to be assigned exactly zero relevance, while the remaining keys are aggregated without competition among keys.

Empirically, Multiscreen improves parameter efficiency, training stability at large learning rates, retrieval ability, and full-context forward-pass latency compared to Transformer baselines, while maintaining stable long-context perplexity and showing little degradation in retrieval performance as context length increases.

More broadly, our findings suggest that improving long-context behavior requires moving beyond redistribution-based mechanisms toward architectures that select information using absolute relevance. This perspective provides a new lens for understanding how models process and utilize context, and may enable more transparent and interpretable analysis of model behavior.

Acknowledgments and Disclosure of Funding

This research was conducted using the Supermicro ARS-111GL-DNHR-LCC and FUJITSU Server PRIMERGY CX2550 M7 (Miyabi) at the Joint Center for Advanced High Performance Computing (JCAHPC), as well as the computer resources offered under the category of General Projects by the Research Institute for Information Technology, Kyushu University.

KMN is supported by the Center of Innovation for Sustainable Quantum AI (JST Grant Number JPMJPF2221).

This work was supported by RIKEN through its institutional funding.

References

- [1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.
- [2] Ken M. Nakanishi. Scalable-Softmax is superior for attention. *arXiv preprint arXiv:2501.19399*, 2025.
- [3] Xuechen Zhang, Xiangyu Chang, Mingchen Li, Amit Roy-Chowdhury, Jiasi Chen, and Samet Oymak. Selective Attention: Enhancing transformer through principled context control. In *Advances in Neural Information Processing Systems*, 2024.
- [4] Zhixuan Lin, Evgenii Nikishin, Xu He, and Aaron Courville. Forgetting Transformer: Softmax attention with a forget gate. In *International Conference on Learning Representations*, 2025.
- [5] André F. T. Martins and Ramón Fernandez Astudillo. From softmax to sparsemax: A sparse model of attention and multi-label classification. In *International Conference on Machine Learning*, 2016.
- [6] Ben Peters, Vlad Niculae, and André F.T. Martins. Sparse sequence-to-sequence models. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019.
- [7] Gonçalo M. Correia, Vlad Niculae, and André F.T. Martins. Adaptively sparse transformers. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019.
- [8] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.
- [9] Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big Bird: Transformers for longer sequences. In *Advances in Neural Information Processing Systems*, 2020.
- [10] Jingyang Yuan, Huazuo Gao, Damai Dai, Junyu Luo, Liang Zhao, Zhengyan Zhang, Zhenda Xie, Yuxing Wei, Lean Wang, Zhiping Xiao, Yuqing Wang, Chong Ruan, Ming Zhang, Wenfeng Liang, and Wangding Zeng. Native Sparse Attention: Hardware-aligned and natively trainable sparse attention. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2025.
- [11] Di Liu, Meng Chen, Baotong Lu, Huiqiang Jiang, Zhenhua Han, Qianxi Zhang, Qi Chen, Chengruidong Zhang, Bailu Ding, Kai Zhang, Chen Chen, Fan Yang, Yuqing Yang, and Lili Qiu. RetrievalAttention: Accelerating long-context LLM inference via vector retrieval. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.
- [12] Jason Ramapuram, Federico Danieli, Eeshan Dhekane, Floris Weers, Dan Busbridge, Pierre Ablin, Tatiana Likhomanenko, Jagrit Digani, Zijin Gu, Amitis Shidani, and Russ Webb. Theory, analysis, and best practices for sigmoid self-attention. In *International Conference on Learning Representations*, 2025.

- [13] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. In *First Conference on Language Modeling*, 2024.
- [14] Michael Poli, Stefano Massaroli, Eric Nguyen, Daniel Y. Fu, Tri Dao, Stephen Baccus, Yoshua Bengio, Stefano Ermon, and Christopher Ré. Hyena hierarchy: Towards larger convolutional language models. In *International Conference on Machine Learning*, 2023.
- [15] Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. Retentive Network: A successor to transformer for large language models. *arXiv preprint arXiv:2307.08621*, 2023.
- [16] Simran Arora, Sabri Eyuboglu, Aman Timalsina, Isys Johnson, Michael Poli, James Zou, Atri Rudra, and Christopher Ré. Zoology: Measuring and improving recall in efficient language models. In *International Conference on Learning Representations*, 2024.
- [17] Ofir Press, Noah Smith, and Mike Lewis. Train short, test long: Attention with linear biases enables input length extrapolation. In *International Conference on Learning Representations*, 2022.
- [18] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.
- [19] Shouyuan Chen, Sherman Wong, Liangjian Chen, and Yuandong Tian. Extending context window of large language models via positional interpolation. *arXiv preprint arXiv:2306.15595*, 2023.
- [20] Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. YaRN: Efficient context window extension of large language models. In *International Conference on Learning Representations*, 2024.
- [21] Yiran Ding, Li Lyna Zhang, Chengruidong Zhang, Yuanyuan Xu, Ning Shang, Jiahang Xu, Fan Yang, and Mao Yang. LongRoPE: Extending LLM context window beyond 2 million tokens. In *International Conference on Machine Learning*, 2024.
- [22] Shanda Li, Chong You, Guru Guruganesh, Joshua Ainslie, Santiago Ontanon, Manzil Zaheer, Sumit Sanghai, Yiming Yang, Sanjiv Kumar, and Srinadh Bhojanapalli. Functional interpolation for relative positions improves long context transformers. In *The Twelfth International Conference on Learning Representations*, 2024.
- [23] Amirhossein Kazemnejad, Inkit Padhi, Karthikeyan Natesan Ramamurthy, Payel Das, and Siva Reddy. The impact of positional encoding on length generalization in transformers. In *Advances in Neural Information Processing Systems*, 2023.
- [24] Jie Wang, Tao Ji, Yuanbin Wu, Hang Yan, Tao Gui, Qi Zhang, Xuan-Jing Huang, and Xiaoling Wang. Length generalization of causal transformers without position encoding. In *Findings of the Association for Computational Linguistics: ACL 2024*, 2024.
- [25] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.
- [26] Alex Graves, Greg Wayne, and Ivo Danihelka. Neural Turing machines. *arXiv preprint arXiv:1410.5401*, 2014.
- [27] Catherine Olsson, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, Tom Henighan, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Scott Johnston, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Ndousse, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, and Chris Olah. In-context learning and induction heads. *arXiv preprint arXiv:2209.11895*, 2022.
- [28] Greg Kamradt. Needle in a haystack - pressure testing LLMs, 2023. Accessed on Jan 19, 2024.

- [29] Amirkeivan Mohtashami and Martin Jaggi. Random-access infinite context length for transformers. In *Advances in Neural Information Processing Systems*, 2023.
- [30] Ken Shi and Gerald Penn. Semantic masking in a needle-in-a-haystack test for evaluating large language model long-text capabilities. In *Proceedings of the First Workshop on Writing Aids at the Crossroads of AI, Cognitive Science and NLP (WRAICOGS 2025)*, pages 16–23, 2025.
- [31] Aaron van den Oord, Nal Kalchbrenner, Oriol Vinyals, Lasse Espeholt, Alex Graves, and Koray Kavukcuoglu. Conditional image generation with PixelCNN decoders. In *Advances in Neural Information Processing Systems*, 2016.
- [32] Yann N Dauphin, Angela Fan, Michael Auli, and David Grangier. Language modeling with gated convolutional networks. In *International Conference on Machine Learning*, 2017.
- [33] Noam Shazeer. GLU variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.
- [34] Goro Kobayashi, Tatsuki Kuribayashi, Sho Yokoi, and Kentaro Inui. Attention is not only a weight: Analyzing transformers with vector norms. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, 2020.
- [35] Zhiyu Guo, Hidetaka Kamigaito, and Taro Watanabe. Attention score is not all you need for token importance indicator in KV cache reduction: Value also matters. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024.
- [36] Stefan Elfving, Eiji Uchibe, and Kenji Doya. Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. *Neural networks*, 107:3–11, 2018.
- [37] Zihan Qiu, Zekun Wang, Bo Zheng, Zeyu Huang, Kaiyue Wen, Songlin Yang, Rui Men, Le Yu, Fei Huang, Suozhi Huang, Dayiheng Liu, Jingren Zhou, and Junyang Lin. Gated attention for large language models: Non-linearity, sparsity, and attention-sink-free. In *Advances in Neural Information Processing Systems*, 2025.
- [38] Daria Soboleva, Faisal Al-Khateeb, Robert Myers, Jacob R Steeves, Joel Hestness, and Nolan Dey. SlimPajama: A 627B token cleaned and deduplicated version of RedPajama. <https://cerebras.ai/blog/slimpajama-a-627b-token-cleaned-and-deduplicated-version-of-redpajama>, 2023.
- [39] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.
- [40] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, 2022.
- [41] Philippe Tillet, H. T. Kung, and David Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, 2019.
- [42] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.
- [43] Maurice Weber, Daniel Fu, Quentin Anthony, Yonatan Oren, Shane Adams, Anton Alexandrov, Xiaozhong Lyu, Huu Nguyen, Xiaozhe Yao, Virginia Adams, Ben Athiwaratkun, Rahul Chalamala, Kezhen Chen, Max Ryabinin, Tri Dao, Percy Liang, Christopher Ré, Irina Rish, and Ce Zhang. RedPajama: an open dataset for training large language models. In *Advances in Neural Information Processing Systems*, 2024.
- [44] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

- [45] Stella Biderman, Hailey Schoelkopf, Quentin Anthony, Herbie Bradley, Kyle O’Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, Aviya Skowron, Lintang Sutawika, and Oskar van der Wal. Pythia: A suite for analyzing large language models across training and scaling. In *International Conference on Machine Learning*, 2023.
- [46] Alexander Hägele, Elie Bakouch, Atli Kosson, Loubna Ben allal, Leandro Von Werra, and Martin Jaggi. Scaling laws and compute-optimal training beyond fixed training durations. In *Advances in Neural Information Processing Systems*, 2024.
- [47] Kaiyue Wen, Zhiyuan Li, Jason Wang, David Hall, Percy Liang, and Tengyu Ma. Understanding Warmup-Stable-Decay learning rates: A river valley loss landscape view. In *International Conference on Learning Representations*, 2025.
- [48] Kazuki Yano, Shun Kiyono, Sosuke Kobayashi, Sho Takase, and Jun Suzuki. Pre-training LLM without learning rate decay enhances supervised fine-tuning. In *International Conference on Learning Representations*, 2026.
- [49] Sidney Black, Stella Biderman, Eric Hallahan, Quentin Anthony, Leo Gao, Laurence Golding, Horace He, Connor Leahy, Kyle McDonell, Jason Phang, Michael Pieler, Usvsn Sai Prashanth, Shivanshu Purohit, Laria Reynolds, Jonathan Tow, Ben Wang, and Samuel Weinbach. GPT-NeoX-20B: An open-source autoregressive language model. In *Proceedings of BigScience Episode #5 – Workshop on Challenges & Perspectives in Creating Large Language Models*, 2022.
- [50] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *North American Chapter of the Association for Computational Linguistics*, 2019.
- [51] Alex Henry, Prudhvi Raj Dachapally, Shubham Shantaram Pawar, and Yuxuan Chen. Query-key normalization for transformers. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020.
- [52] Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar, Chloe Hillier, and Timothy P. Lillicrap. Compressive transformers for long-range sequence modelling. In *International Conference on Learning Representations*, 2020.
- [53] Arize AI. Needle in a haystack - pressure testing LLMs, 2023. Accessed on Jan 19, 2024.
- [54] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations*, 2021.
- [55] Karl Cobbe, Vineet Kosaraju, Mo Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- [56] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoît Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, 2019.

A Experimental Setup

We compare Multiscreen against a Transformer baseline under matched training data and token budgets.

Tokenization. We adopt the GPT-2 tokenizer [42] with a vocabulary of 50,257 tokens.

Training Data. We pretrain all models on the SlimPajama [38] dataset, a compressed version of the RedPajama [43] dataset. After tokenization, documents are concatenated with EOS tokens to form a continuous token stream of approximately 628 billion tokens. We use 2^{38} tokens (approximately 44% of the dataset), obtained by partitioning the tokenized corpus into contiguous sequences of length 2^{12} and selecting 2^{26} sequences at random.

Model Sizes. To analyze scaling behavior, we train both Transformer and Multiscreen models across multiple parameter scales.

For the Transformer baseline, we adopt a LLaMA-style architecture [44], with architecture hyperparameters based on those used in Pythia [45], and apply weight tying between the input embedding and the language modeling head. We follow Pythia configurations where applicable, including model sizes and learning rate schedules. Detailed configurations of the Transformer baseline are provided in Appendix C.

For Multiscreen, the architecture hyperparameters are summarized in Appendix B.

Training Setup. All models are pretrained using 2^{38} tokens with a sequence length of 2^{12} (4,096). We use a global batch size of 2^{22} tokens.

All models are optimized using AdamW [39] with $(\beta_1, \beta_2) = (0.9, 0.95)$. We use 2^{12} warmup steps, after which the learning rate is kept constant. This warmup-then-constant schedule keeps the optimization protocol simple and comparable across architectures under a fixed token budget, and is consistent with recent work studying long stable or decay-free learning-rate phases [46, 47, 48].

For Transformer, we use RoPE with $\theta = 10,000$, and adopt a training configuration based on Pythia [45], including weight decay (0.1), gradient clipping (threshold 1.0), and model-size-dependent learning rates. The learning rate is set to the peak value used in Pythia for each model scale: 1×10^{-3} for 8M and 45M models, 3×10^{-4} for 353M, and 2×10^{-4} for 1.3B.

For Multiscreen, we use the same optimizer configuration as the Transformer baseline, but omit weight decay and gradient clipping, which we found unnecessary for stable training in our experiments. We use a learning rate of 2^{-4} (i.e., 0.0625), substantially larger than those used for Transformer, enabled by the improved stability of Multiscreen demonstrated in section 4.3.

Optimizer-setting sanity check. As a preliminary check, we trained additional 45M Transformer baselines under two modified optimizer settings: one with weight decay removed, and one with gradient clipping disabled. Each setting was evaluated over three random seeds. Removing weight decay slightly improved validation loss by approximately 0.02, while disabling gradient clipping produced nearly indistinguishable loss curves. These results suggest that the main scaling conclusions are not driven by these optimizer details, although we did not repeat this check at every model scale.

B Multiscreen Architecture and Scaling

We scale Multiscreen using a single supraparameter Ψ that jointly determines the number of layers, the number of heads, and the embedding dimension. In our experiments, the number of layers and the number of heads are set to $N_L = N_H = \Psi$, and the embedding dimension is set to $d_E = \Psi^2$.

Other architectural hyperparameters are kept fixed across model scales, as summarized in table 1. The resulting model configurations and parameter counts across scales are shown in table 2.

Table 1: Architectural hyperparameters of Multiscreen used in our experiments.

Hyperparameter	Symbol	Value / scaling
Number of layers	N_L	Ψ
Number of heads	N_H	Ψ
Embedding dimension	d_E	Ψ^2
Key dimension	d_K	16
Value dimension	d_V	64
MiPE threshold	w_{th}	256

Table 2: Architecture configurations and parameter counts for the Multiscreen models used in the scaling experiments.

Hyperparameter	4M	28M	286M	1.3B	4B
Supraparameter (Ψ)	8	16	32	48	64
Total params	4,134,146	27,546,626	286,347,266	1,304,884,226	3,963,961,346
Non-embedding params	917,698	14,680,834	234,884,098	1,189,092,098	3,758,108,674

C Transformer Baseline Configurations

We describe the architecture configurations of the Transformer baseline models used in our experiments. The Transformer baseline adopts a LLaMA-style architecture [44], with architecture hyperparameters largely aligned with those used in Pythia [45]. We apply weight tying between the input embedding and the language-modeling head, which differs from the original Pythia configuration, to improve parameter efficiency and strengthen the baseline.

The head dimension is defined as d_E/N_H , and the feed-forward dimension is set to $\lfloor \frac{8}{3}d_E \rfloor$. The architecture configurations across model scales are summarized in table 3.

Initialization. For the Transformer baseline, we follow the Pythia/GPT-NeoX initialization scheme [45, 49]. Specifically, all weights except residual output projections are initialized using small initialization with standard deviation $\sqrt{2/(5d_E)}$, while residual output projections (i.e., the output projections of the attention and feed-forward blocks) are initialized using Wang initialization with standard deviation $2/(N_L\sqrt{d_E})$. This scheme performed better in our preliminary experiments than fixed-scale Gaussian initialization (standard deviation 0.02) with residual scaling by $1/\sqrt{2N_L}$, which we used in an earlier version of this work following common Transformer practice [50, 42].

Table 3: Architecture hyperparameters for the Transformer baseline models across different parameter scales.

Hyperparameter	8M	45M	353M	1.3B
Number of layers (N_L)	6	6	24	24
Number of heads (N_H)	4	8	16	16
Embedding dimension (d_E)	128	512	1,024	2,048
Total params	7,613,312	44,609,024	353,453,056	1,310,935,040
Non-embedding params	1,180,416	18,877,440	301,989,888	1,208,008,704

D Multiscreen Initialization

We initialize all projection matrices and the embedding matrix from zero-mean Gaussian distributions whose standard deviations are scaled inversely with the square root of their output dimension (i.e., the second dimension of each weight matrix), as summarized in table 4. The gate projection W_G is initialized with a fixed scale independent of dimension. For matrices whose outputs are subsequently unit-normalized, the global initialization scale does not affect the norm of the normalized vectors, but can still affect early optimization dynamics; for projections not followed by such normalization, it directly affects the initial activation scale.

The window parameter s_w is initialized independently in each layer with values linearly spaced across heads from 0 to $\log w_{th}$. The parameter s_r is initialized to zero, corresponding to an initial acceptance width of $r = 0.5$. The residual output scale s_O is initialized to approximately normalize the total contribution across all tiles in the model, ensuring that the aggregated residual updates remain well-scaled as the number of tiles increases.

The parameters s_E and s_F control the scale of input embeddings and output logits, respectively, and are initialized to set appropriate scales for these quantities.

Table 4: Parameter shapes and initialization of Multiscreen.

Parameter	Shape	Initialization
W_Q	(d_E, d_K)	$\mathcal{N}(0, 0.1/\sqrt{d_K})$
W_K	(d_E, d_K)	$\mathcal{N}(0, 0.1/\sqrt{d_K})$
W_V	(d_E, d_V)	$\mathcal{N}(0, 0.1/\sqrt{d_V})$
W_G	(d_E, d_V)	$\mathcal{N}(0, 0.1)$
W_O	(d_V, d_E)	$\mathcal{N}(0, 0.1/\sqrt{d_E})$
W_E	$(\mathcal{V} , d_E)$	$\mathcal{N}(0, 0.1/\sqrt{d_E})$
s_w	scalar	linearly spaced across heads from 0 to $\log w_{th}$ in each layer
s_r	scalar	0
s_O	scalar	$\log(1/\sqrt{N_H N_L})$
s_E	scalar	0
s_F	scalar	$\log \sqrt{d_E}$

E Additional Scaling Analysis

We further analyze scaling behavior using alternative definitions of model size, as shown in fig. 9.

Non-embedding parameters. In the left panel of fig. 9, we plot validation loss as a function of non-embedding parameters for both Transformer and Multiscreen. The same trend is observed, with an even clearer approximately linear scaling trend, and the relative advantage of Multiscreen over Transformer is preserved under this parameterization.

Supraparameter Ψ . In the right panel of fig. 9, we analyze scaling behavior using a unified supraparameter Ψ defined for Multiscreen. This parameterization also yields a clean scaling relationship, suggesting that Ψ provides a compact and unified characterization of model scaling for Multiscreen.

In both parameterizations, the 4B Multiscreen model deviates from the fitted scaling trend, which we attribute to undertraining at this scale. All models are trained with a fixed token budget, and larger models typically require more tokens to reach comparable convergence, suggesting that this deviation would diminish with additional training.

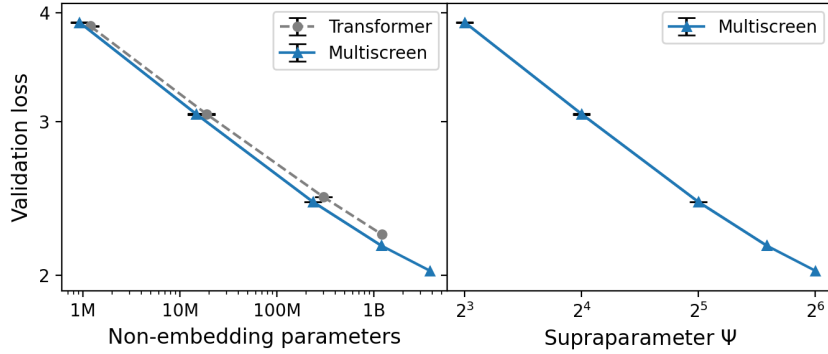


Figure 9: Scaling behavior under alternative definitions of model size. Left: scaling behavior of Transformer and Multiscreen with respect to non-embedding parameters. Right: scaling behavior of Multiscreen with respect to the supraparameter Ψ . The 4B Multiscreen model deviates from the scaling trend, which we attribute to undertraining.

F Learning Rate Sweep Details

We provide additional details for the learning-rate sweep experiments shown in fig. 5.

Setup. We evaluate training stability by sweeping the learning rate over a logarithmically spaced grid ranging from 2^{-14} to 2^0 . All runs use the same training configuration as described in section 4.1, with the learning rate as the only varying parameter.

Models. We use a 45M-parameter Transformer and a 28M-parameter Multiscreen model. These sizes are chosen to balance computational cost with clear visibility of instability effects.

Evaluation. Training stability is assessed based on validation loss. For each learning rate, we train three independent runs with different random seeds and report the mean and standard deviation. Runs that diverge are excluded from the plot in fig. 5.

Reproducibility. We observe consistent qualitative behavior across random seeds, with Transformer exhibiting instability at moderately large learning rates, while Multiscreen remains stable over a substantially wider range.

G Training Loss Dynamics

To complement the learning-rate sweep in fig. 5, we visualize training-loss trajectories from the same training runs, showing one representative run (using a fixed random seed) for each learning rate.

As shown in fig. 10, Transformer exhibits increasingly unstable behavior as the learning rate increases. At moderate learning rates, training becomes increasingly unstable, with frequent spikes in the loss, and at larger values the optimization fails to converge. In contrast, Multiscreen exhibits stable and smooth convergence even at large learning rates. Notably, even at very large learning rates (e.g., 1), Multiscreen continues to train reliably without divergence.

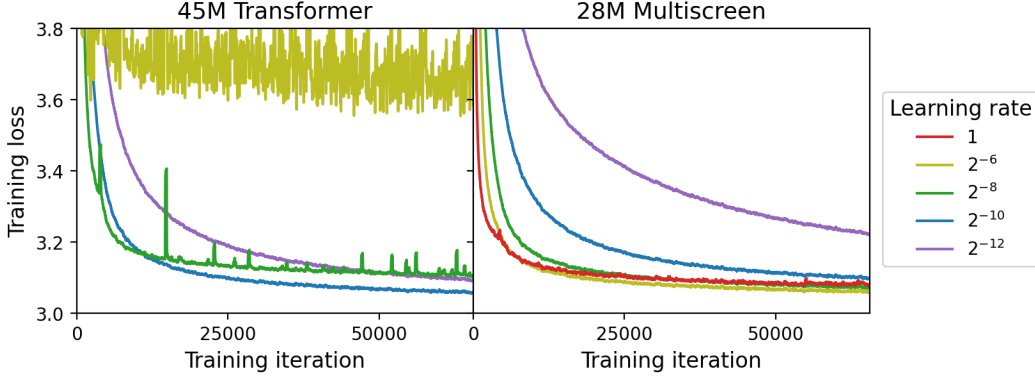


Figure 10: Training loss trajectories from the same runs as in fig. 5, shown for representative learning rates. Curves are smoothed using a moving average over 256 training steps. Transformer becomes unstable at moderately large learning rates, exhibiting increasingly unstable and eventually divergent behavior, whereas Multiscreen maintains stable convergence even at very large learning rates (e.g., 1).

These results provide a complementary view to the validation-based learning-rate sweep. While fig. 5 summarizes the final performance, the training trajectories shown here illustrate how instability emerges during optimization. To better understand these behaviors, we next analyze the gradient dynamics in Appendix H. Together, these observations suggest that Multiscreen exhibits more stable gradient dynamics, leading to more robust optimization and enabling the use of substantially larger learning rates.

H Gradient Norm Dynamics

We report gradient norms from the same training runs used in our scaling experiments (section 4.2), focusing on 1.3B Transformer and 1.3B Multiscreen models. We visualize gradient norms from a single training run at the 1.3B scale for each architecture. We observe qualitatively consistent behavior across different model sizes, and therefore present this run as a representative example.

As shown in fig. 11, Multiscreen exhibits rapidly decaying gradient norms that settle near zero with minimal variance, while Transformer maintains a non-zero gradient floor with substantial variance and occasional spikes.

These observations are consistent with the absence of competition across keys, and with the improved stability of Multiscreen under large learning rates.

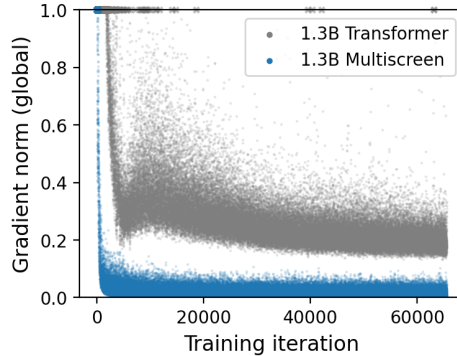


Figure 11: Gradient norm dynamics during training for Transformer and Multiscreen. Multiscreen exhibits rapidly decaying gradient norms that settle near zero with minimal variance, while Transformer maintains a non-zero gradient floor with substantial variance and occasional spikes. For visualization, values above 1 are clipped and shown with $\times$ markers.

Optimizer-setting sanity check. Because the Transformer baseline uses weight decay and gradient clipping whereas Multiscreen omits both, one possible concern is that the gradient-norm comparison is driven by these optimizer settings. As a sanity check, we compare 45M Transformer variants with weight decay removed or gradient clipping disabled. As shown in fig. 12, disabling gradient clipping produces gradient-norm dynamics similar to the standard Transformer baseline, while removing weight decay increases the Transformer gradient norm. In contrast, Multiscreen maintains much smaller gradient norms despite using no weight decay or gradient clipping.

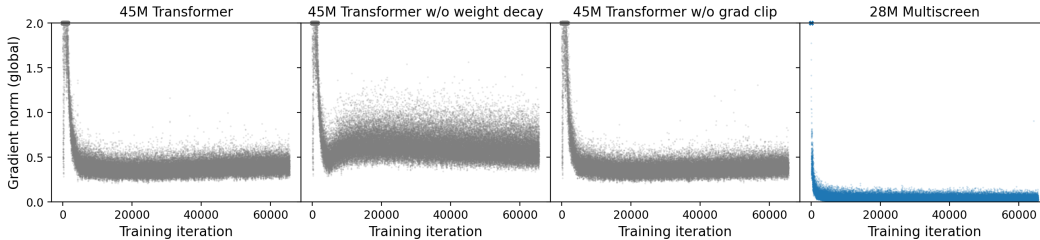


Figure 12: Optimizer-setting sanity check for gradient-norm dynamics. We compare the standard 45M Transformer baseline, 45M Transformer variants without weight decay or gradient clipping, and the 28M Multiscreen model. Removing weight decay increases the Transformer gradient norm, while disabling gradient clipping leaves the dynamics similar to the standard Transformer. Multiscreen maintains substantially smaller gradient norms despite using no weight decay or gradient clipping.

I QKNorm Transformer Baseline

Multiscreen applies unit-length normalization to queries and keys before computing query–key similarities. One possible concern is that the improved learning-rate stability of Multiscreen may be explained primarily by this normalization rather than by screening itself. This concern is natural because query–key normalization (QKNorm) has been proposed to make softmax attention less prone to arbitrary saturation by normalizing queries and keys before computing attention scores [51]. To test this, we evaluate an additional Transformer baseline with QKNorm. In our implementation, QKNorm is RMSNorm-based: RMSNorm with elementwise affine parameters is applied to the query and key vectors before computing attention scores, while the rest of the Transformer architecture and training setup are kept unchanged.

As shown in fig. 13a, QKNorm slightly improves validation loss over the standard Transformer baseline at moderate learning rates, but does not remain stable at large learning rates. The training-loss plot in fig. 13b uses the same axes as fig. 10, enabling direct comparison with the standard Transformer and Multiscreen dynamics. While QKNorm mildly stabilizes Transformer training dynamics in some settings, it still exhibits loss spikes at large learning rates and diverges for learning rates $\geq 2^{-4}$.

These results suggest that normalizing queries and keys alone does not explain the learning-rate robustness of Multiscreen.

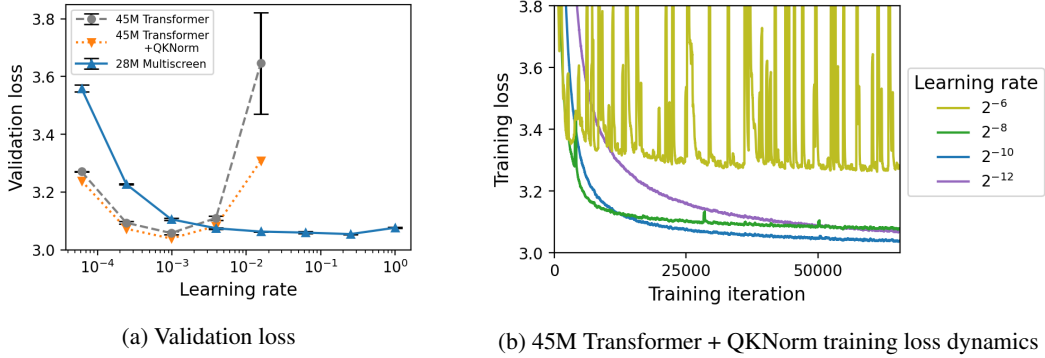


Figure 13: QKNorm Transformer diagnostic experiment. (a) Learning-rate sweep with an additional 45M Transformer + QKNorm baseline. Standard Transformer and Multiscreen results are reproduced from fig. 5 and averaged over three seeds, with error bars indicating the standard deviation across seeds. QKNorm Transformer results are from a single seed and are shown without error bars. (b) Training loss dynamics of the 45M Transformer + QKNorm baseline, plotted with the same axes as fig. 10 for direct comparison. QKNorm slightly stabilizes Transformer training in some settings, but the model still exhibits loss spikes at large learning rates and diverges for learning rates $\geq 2^{-4}$. Thus, query–key normalization alone does not reproduce the large-learning-rate stability of Multiscreen.

J Visualization of Distance-Aware Relevance

We visualize distance-aware relevance maps α_{ij}^d for all tiles in a 4M Multiscreen model used in section 4.2, as shown in fig. 14. These visualizations highlight the presence of learned screening windows and the extent of sparsity within them.

The input sequence is a 203-token natural-language passage (the abstract of this paper). All layers and heads are shown, arranged in a grid layout where rows correspond to layers and columns correspond to heads.

Within each map, dark gray regions indicate positions outside the learned screening window, while color intensity represents the magnitude of α_{ij}^d within the window. Each tile is annotated with its layer and head indices, the learned window width w , the acceptance width r , and the fraction of nonzero relevance values within the window, $\Pr(\alpha_{ij}^d > 0)$.

Different tiles cover different context ranges, with some focusing on local neighborhoods and others covering broader portions of the sequence. Many maps contain a substantial number of zero entries, with the degree of sparsity varying across tiles.

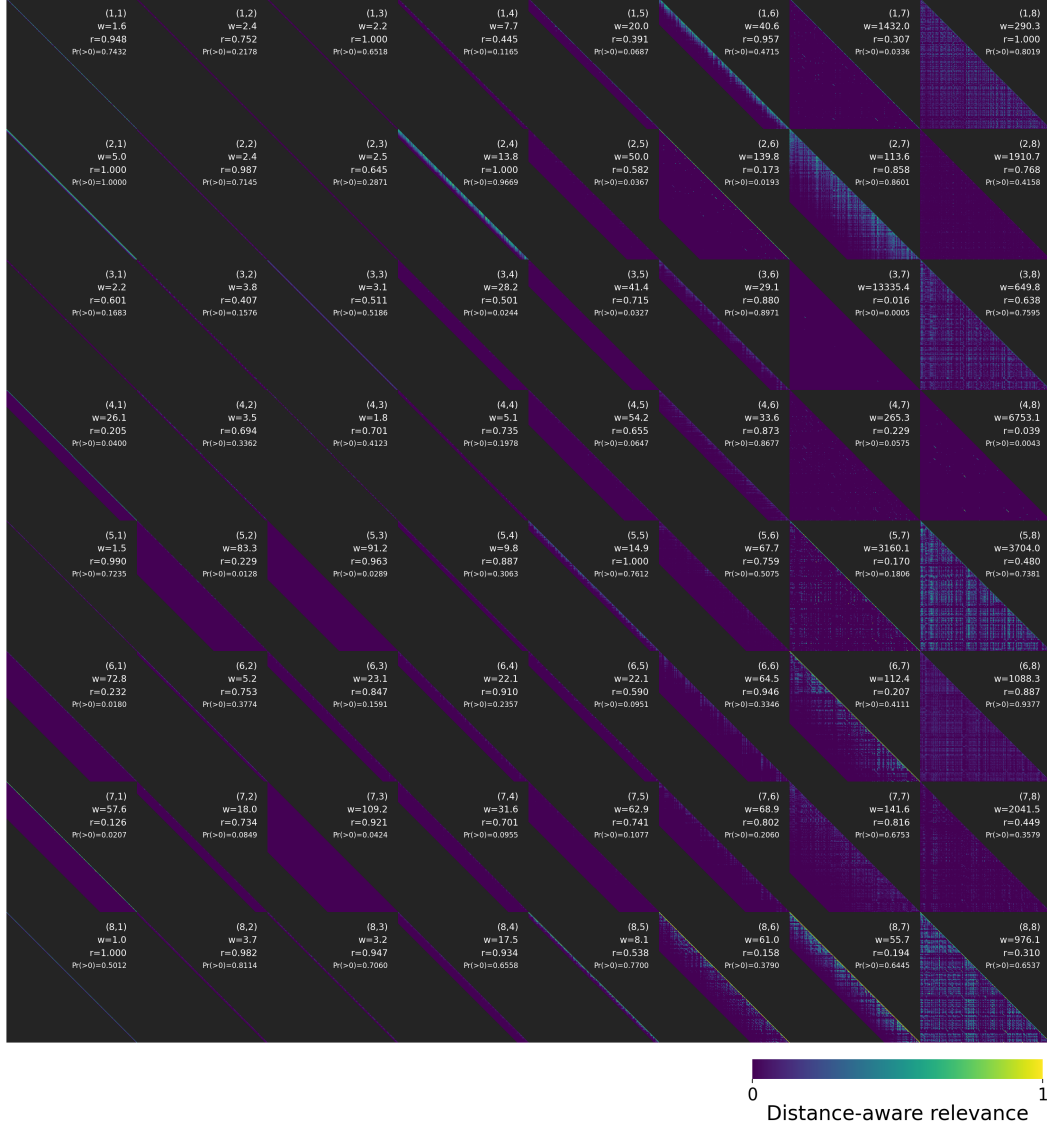


Figure 14: Distance-aware relevance maps across layers and heads. Each map shows the distance-aware relevance α_{ij}^d , with rows and columns corresponding to query and key positions. Darker gray regions indicate positions outside the learned screening window. Each tile is annotated with its layer and head indices, the learned window width w , the acceptance width r , and the fraction of nonzero relevance values $\Pr(\alpha_{ij}^d > 0)$ within the window, providing a summary of its sparsity and coverage.

K Distribution of Learned Screening Parameters

Each screening unit has independently learned scalar parameters s_w and s_r , which determine the screening window w and acceptance width r . To better understand how these parameters are used after training, we visualize the resulting values of w and r across all layers and heads of the 4B Multiscreen model.

As shown in fig. 15, learned screening windows span several orders of magnitude. In this model, most units remain local or medium-range, while a smaller number of units learn very large finite windows, including windows exceeding the pretraining context length. The learned acceptance widths are often close to one, but also vary substantially across units, indicating that different screening units specialize to different degrees of selectivity. This diversity suggests that Multiscreen combines many local screening units with a smaller number of broad-context units.

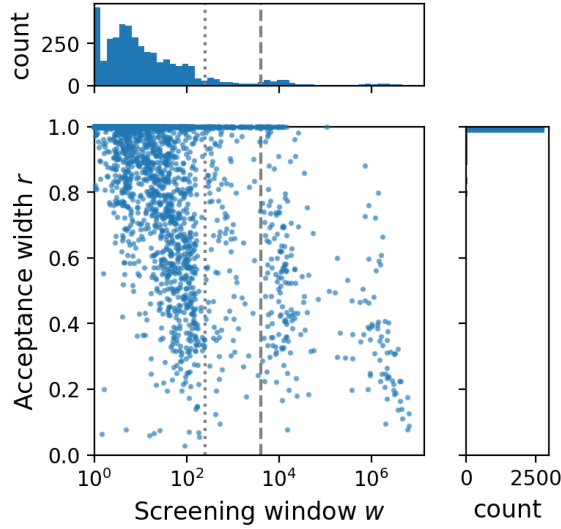


Figure 15: Distribution of learned screening parameters in the 4B Multiscreen model. Each point corresponds to one screening unit, with horizontal position indicating the learned screening window w and vertical position indicating the learned acceptance width r . The horizontal axis is shown on a logarithmic scale. Marginal histograms show the corresponding distributions of w and r . The dotted vertical line marks the MiPE threshold ($w_{th} = 2^8$), above which MiPE becomes inactive, and the dashed vertical line marks the pretraining context length (2^{12}). Learned windows span several orders of magnitude, with many local or medium-range units and a smaller number of very large finite-window units.

L Long-Context Perplexity: Evaluation Details

We provide additional details for the long-context perplexity evaluation in section 4.4, along with robustness results using smaller models trained with multiple random seeds.

Evaluation setup. We evaluate long-context language modeling using position-dependent perplexity over long sequences. Perplexity at each position is averaged within a centered window spanning $\pm 10\%$ of the context length to reduce local variance.

We use the PG-19 dataset [52], consisting of books published before 1919 from Project Gutenberg.² From the dataset, we select 5,747 documents whose tokenized length exceeds 2^{17} . For each document, we extract a contiguous segment of $2^{17} + 1$ tokens centered at the midpoint of the document. This construction ensures that predictions are evaluated in the middle of long contexts rather than near document boundaries. We did not explicitly deduplicate PG-19 against SlimPajama, so overlap between the pretraining corpus and evaluation set may remain possible. However, all models are trained and evaluated under the same conditions, so the relative comparison between Transformer and Multiscreen remains controlled, although absolute perplexity values should be interpreted with this caveat in mind.

RoPE scaling. For Transformer, we evaluate multiple RoPE scaling factors to assess extrapolation beyond the training context length. For base models trained with sequence length 2^{12} , we evaluate scaling factors $\times 1$, $\times 2$, $\times 4$, $\times 8$, $\times 16$, $\times 32$, and $\times 64$. Multiscreen does not use RoPE; its long-context behavior is therefore evaluated without a RoPE scaling factor.

Seed-averaged smaller-model results. In addition to the main long-context perplexity results, we evaluate the 353M Transformer and 286M Multiscreen models used in the scaling experiments. For each architecture, we use three independently trained models with different random seeds and report the mean and one standard deviation.

As shown in fig. 16, the same qualitative behavior as in section 4.4 is observed. Multiscreen maintains stable perplexity as context length increases, whereas Transformer exhibits sharp degradation once the context exceeds the training range. For Transformer, increasing the RoPE scaling factor delays the onset of degradation, but also increases perplexity overall.

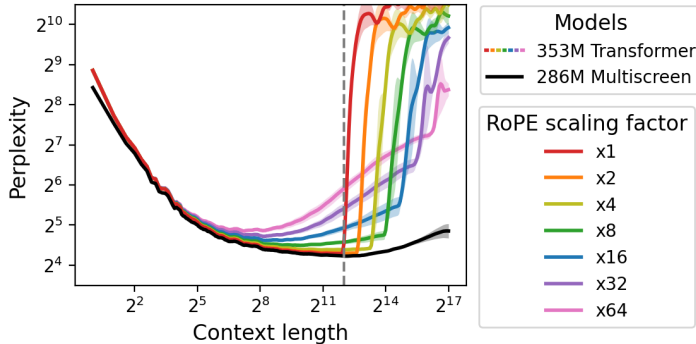


Figure 16: Long-context perplexity comparison for 353M Transformer and 286M Multiscreen models. Curves show mean perplexity over three independently trained models, and shaded regions indicate one standard deviation. The horizontal axis indicates the context length at which position-dependent perplexity is evaluated, and the vertical axis is perplexity. The dashed vertical line marks the pretraining context length (2^{12}). For Transformer, colored curves correspond to different RoPE scaling factors. Multiscreen maintains stable perplexity beyond the training context, while Transformer degrades sharply once the context exceeds the training range.

²<https://huggingface.co/datasets/Geralt-Targaryen/pg19>

M ABCDigits: Task Construction and Evaluation Details

We provide additional details on the construction and evaluation protocol of ABCDigits, the synthetic key-value retrieval benchmark used in section 4.5.

Motivation. ABCDigits builds on synthetic associative recall and key-value retrieval benchmarks studied in prior work [26, 27, 25, 16]. We cast retrieval as a structured completion problem that directly requires recovering the associated value. This provides a direct probe of retrieval behavior while reducing confounding effects from natural-language semantics, instruction following, and prompt-specific factors.

Task construction. Each ABCDigits instance presents a shuffled list of equations mapping uppercase letters to six-digit integers, such as $A=967892$. A query is formed by appending a target letter followed by an equals sign, such as $L=$, and the model must complete the corresponding integer. The target mapping appears exactly once in the context, requiring the model to locate this unique occurrence. Since each letter is consistently mapped to a single integer across the context, the task implicitly enforces a one-to-one key-value correspondence without requiring explicit instructions.

A key design choice is that the number of distinct keys remains fixed at 26 uppercase letters regardless of context length. This removes the bias that retrieval becomes harder simply because the number of keys increases, allowing us to better isolate length-dependent retrieval behavior.

To ensure that extremely low-frequency equations are not anomalous within the context, we construct the non-target portion of the context in two stages. First, we include exactly one instance of each of the 25 non-target letter-digit equations. We then fill the remaining context by sampling additional non-target equations from a highly skewed categorical distribution with weights proportional to $2^0, 2^1, \dots, 2^{24}$, where the assignment from letters to weights is randomized for each instance. This avoids making infrequent key-value pairs appear anomalous, ensuring that a range of frequencies is naturally represented within the context. All non-target equations are then shuffled. Finally, the unique target equation, such as $L=169428$, is inserted at a specified depth, and the query suffix $L=$ is appended. A concrete example of the resulting prompt is shown in fig. 7a in the main text.

Evaluation protocol. Following the visualization protocol commonly used in needle-in-a-haystack evaluations [28, 53], we measure retrieval accuracy across a grid of context lengths and target depths. Context lengths range from 2^{12} to 2^{17} , and depths are set to 0.1, 0.3, 0.5, 0.7, and 0.9. Depth is implemented at the equation level: a depth of d means that the target equation is inserted after approximately a fraction d of the equations in the context, before the final query.

For each context length and depth, we generate 1,000 independent ABCDigits instances. We evaluate exact-match accuracy under greedy decoding with temperature 0. An instance is counted as correct only if the generated integer exactly matches the target value.

Models and RoPE scaling. In the main ABCDigits evaluation, we compare 1.3B Transformer, 286M Multiscreen, and 1.3B Multiscreen models. For the 1.3B Transformer and 1.3B Multiscreen models, we evaluate a single trained model for each architecture. For the 286M Multiscreen model, results are averaged over three independently trained models, with 1,000 instances evaluated per model and setting.

For Transformer, we evaluate multiple RoPE scaling factors to assess retrieval under different positional extrapolation settings. For each context length and depth, we report the best average accuracy across the candidate RoPE scaling factors, and the selected factor is shown in each cell of fig. 7. For Multiscreen, no RoPE scaling factor is used.

Additional seed-averaged and scaling results. To further assess robustness and scaling behavior, we report ABCDigits retrieval accuracy across a range of model sizes for both Transformer and Multiscreen, as shown in fig. 17.

For models up to 353M (Transformer) and 286M (Multiscreen), results are averaged over three independently trained models, and we report the mean accuracy across seeds. For the 1.3B models, a single trained model is evaluated due to computational constraints.

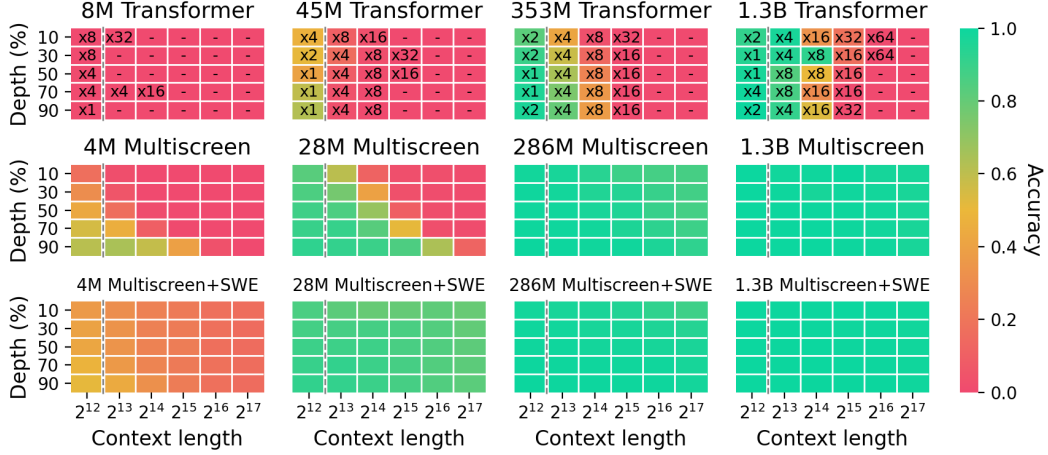


Figure 17: ABCDigits retrieval accuracy across model sizes for Transformer and Multiscreen. Top row: Transformer baselines. Middle row: Multiscreen models using learned screening windows without expansion. Bottom row: Multiscreen models with inference-time screening-window expansion (SWE), where learned screening windows exceeding the training context length are set to $w = \infty$. For models up to 353M (Transformer) and 286M (Multiscreen), results are averaged over three independently trained models. For the 1.3B models, a single run is shown. For Transformer, each cell reports accuracy under the best-performing RoPE scaling factor selected from multiple candidates. Multiscreen maintains substantially higher retrieval accuracy across context lengths, with only minor degradation for larger models, while Transformer degrades rapidly beyond the training context despite selecting the best RoPE scaling factor. SWE improves long-context retrieval for smaller Multiscreen models, while larger Multiscreen models already perform strongly without it.

The results show a consistent qualitative trend across model sizes. Multiscreen maintains substantially higher retrieval accuracy across context lengths, with only minor degradation for larger models. While very small Multiscreen models exhibit some degradation at longer contexts, the 286M and 1.3B models show little degradation as context length increases. In contrast, Transformer performance degrades rapidly as context length increases. Although moderate extrapolation can be partially mitigated by RoPE scaling, performance deteriorates sharply beyond this regime and eventually collapses, with no successful retrieval at sufficiently long contexts, despite selecting the best RoPE scaling factor for each setting.

Notably, Multiscreen outperforms the corresponding Transformer baseline at every scale even at the training context length, despite using fewer parameters, highlighting a substantial gap in retrieval ability. These results indicate that the strong retrieval performance of Multiscreen is robust across model sizes and random seeds, and that the gap between Multiscreen and Transformer persists across scales.

Effect of inference-time screening-window expansion. We also evaluate optional inference-time screening-window expansion (SWE), where learned screening windows exceeding the training context length are set to $w = \infty$. The main results use the learned screening windows without expansion. As shown in the bottom row of fig. 17, SWE improves long-context retrieval for smaller Multiscreen models, especially the 4M and 28M models. In contrast, larger Multiscreen models already maintain high retrieval accuracy without SWE, suggesting that their retrieval performance is not primarily explained by this inference-time expansion.

N Passkey Retrieval Results

We evaluate retrieval performance on the standard passkey retrieval benchmark [29], as a complementary test to ABCDigits.

Setup. The experimental setup follows that of ABCDigits (section M). We use the same range of context lengths (2^{12} to 2^{17}), target depths (0.1 to 0.9), and number of trials (1,000 instances per setting). The same model scales and RoPE-scaling protocol are used, and the only difference is the prompt format, which follows the passkey retrieval task.

Results. The results are shown in fig. 18. We observe the same qualitative trend as in ABCDigits: Multiscreen is substantially more robust to increasing context length than Transformer. With learned screening windows used without expansion, larger Multiscreen models maintain high retrieval accuracy, while smaller models exhibit some degradation at longer contexts.

In contrast, Transformer performance degrades rapidly as context length increases. Although moderate extrapolation can be partially mitigated by RoPE scaling, performance deteriorates sharply beyond this regime and eventually collapses, despite selecting the best RoPE scaling factor for each setting. As in ABCDigits, Multiscreen outperforms the corresponding Transformer baseline at every scale even at the training context length, despite using fewer parameters.

Effect of inference-time screening-window expansion. We also evaluate inference-time screening-window expansion (SWE), following the definition in section M. As shown in the bottom row of fig. 18, SWE substantially improves long-context passkey retrieval for smaller Multiscreen models. With SWE, even the 28M Multiscreen model achieves over 99.5% accuracy at every evaluated context length and target depth. The 286M and 1.3B Multiscreen models achieve perfect accuracy across all evaluated settings. These results show that the improved retrieval ability of Multiscreen is not specific to the ABCDigits construction. They also show that SWE can further improve long-context coverage in smaller models on a standard retrieval benchmark.

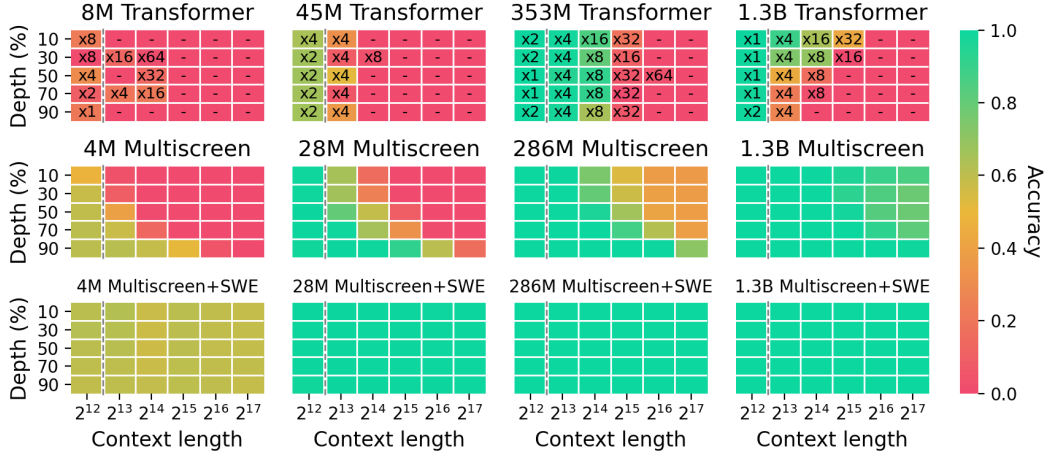


Figure 18: Passkey retrieval accuracy across model sizes for Transformer and Multiscreen. The setup follows that of ABCDigits, with identical context lengths, target depths, and evaluation protocol. Top row: Transformer baselines. Middle row: Multiscreen models using learned screening windows without expansion. Bottom row: Multiscreen models with inference-time screening-window expansion (SWE), where learned screening windows exceeding the training context length are set to $w = \infty$. For Transformer, each cell reports accuracy under the best-performing RoPE scaling factor selected from multiple candidates. Multiscreen is more robust to increasing context length than Transformer, and SWE substantially improves long-context retrieval for smaller Multiscreen models. With SWE, the 28M Multiscreen model exceeds 99.5% accuracy in every cell, and the 286M and 1.3B Multiscreen models achieve perfect accuracy across the grid.

O Latency Impact of Screening-Window Expansion

In Appendices M and N, we evaluate inference-time screening-window expansion (SWE), an optional inference-time rule that sets learned screening windows exceeding the training context length to $w = \infty$. SWE improves long-context retrieval coverage for smaller Multiscreen models, but may increase latency because fewer terms are skipped by the screening-window implementation. Here, we measure this tradeoff using the same full-context forward-pass setup as in section 4.6.

As shown in fig. 19, SWE increases latency relative to default Multiscreen inference. In the model-size sweep, the SWE curve generally lies between the default Multiscreen and Transformer curves over the overlapping model-size range. In the context-length sweep, the 1.3B Multiscreen model with SWE remains faster than the 1.3B Transformer across all evaluated context lengths. Thus, SWE provides an optional retrieval-oriented inference setting that improves long-context coverage while preserving much of Multiscreen’s latency advantage.

Importantly, the main latency results in section 4.6 use default Multiscreen inference without SWE, so the latency improvements reported in the main text do not rely on this optional expansion rule.

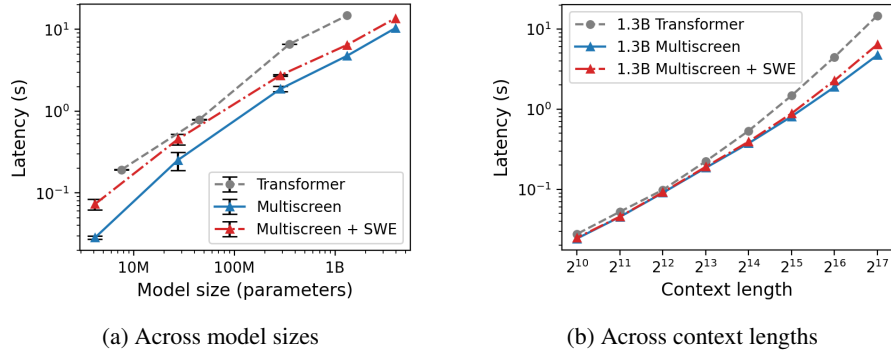


Figure 19: Latency impact of inference-time screening-window expansion (SWE). Measurements use the same full-context forward-pass setup as in section 4.6. (a) Latency vs. model size at context length 2^{17} . Error bars indicate standard deviation across independently trained models, shown for smaller models. (b) Latency vs. context length for 1.3B Transformer, 1.3B Multiscreen, and 1.3B Multiscreen + SWE models. SWE increases latency relative to default Multiscreen inference because fewer terms are skipped, but Multiscreen with SWE preserves much of the latency advantage over Transformer.

P Architectural Ablations

We conduct ablation studies to better understand the contribution of individual Multiscreen components. All ablations in this section are based on the 28M Multiscreen configuration and are trained from scratch with the same optimizer settings as the original Multiscreen model. To reduce computational cost, these ablation runs are trained for 2^{36} tokens rather than the 2^{38} tokens used in the main experiments. The “Original” baseline in this section is therefore also evaluated at the 2^{36} -token checkpoint. Each variant is trained with three random seeds.

We consider five ablations. First, we replace MiPE with RoPE or NoPE to assess the role of positional encoding. For the RoPE ablation, we also evaluate RoPE scaling factors when evaluating beyond the training context. Second, we remove TanhNorm to assess the role of output-norm control after unnormalized aggregation. Third, we remove the gate in the gated screening tile. Because removing the gate reduces the parameter count, we increase both the number of layers and the number of heads from 16 to 19 while keeping the embedding dimension fixed; this makes the no-gate model slightly larger than the original model (27,653,437 parameters vs. 27,546,626). Finally, we fix the acceptance width to $r = 1$ to assess the importance of learning the Trim acceptance width.

Validation loss. Validation losses are shown in table 5. All ablations increase validation loss relative to the original model. Removing the gate produces the largest degradation, despite the no-gate model having slightly more parameters. This suggests that the gate contributes substantially to standard language-modeling performance. Replacing MiPE with NoPE, removing TanhNorm, and fixing $r = 1$ lead to smaller but consistent increases in validation loss. Replacing MiPE with RoPE also worsens validation loss, indicating that applying conventional RoPE-style rotations in this architecture is not beneficial even within the standard validation setting.

Table 5: Validation loss for 28M Multiscreen ablations. Values are averaged over three independently trained models, with standard deviation shown after $\pm$. All models are trained for 2^{36} tokens.

Variant	Validation loss $\downarrow$
Original	3.0966 $\pm$ 0.0027
MiPE $\rightarrow$ RoPE	3.1232 $\pm$ 0.0014
MiPE $\rightarrow$ NoPE	3.1109 $\pm$ 0.0025
w/o TanhNorm	3.1094 $\pm$ 0.0026
w/o Gate	3.1555 $\pm$ 0.0007
Fixed $r = 1$	3.1120 $\pm$ 0.0026

Long-context perplexity. We next evaluate long-context perplexity for the same ablations. As shown in fig. 20, the original model maintains stable perplexity beyond the training context. Replacing MiPE with RoPE degrades long-context perplexity, and the degradation persists across representative RoPE scaling factors. In contrast, replacing MiPE with NoPE has a much smaller effect on perplexity, suggesting that the long-context perplexity gains of Multiscreen are not simply explained by conventional positional extrapolation. However, the original MiPE model achieves the best validation loss and strong retrieval performance, indicating that MiPE provides useful positional structure without forcing large-window tiles to extrapolate RoPE-like rotations.

The fixed- r ablation shows the clearest long-context perplexity degradation, with perplexity increasing sharply at long contexts. This indicates that learning the acceptance width in Trim is important for robust long-context behavior. Removing TanhNorm has a smaller effect on long-context perplexity than on retrieval accuracy, suggesting that perplexity alone does not fully reflect its role in controlled retrieval. Removing the gate has only a modest effect on long-context perplexity, despite substantially degrading validation loss.

Retrieval accuracy. Finally, we evaluate ABCDigits retrieval accuracy for the same ablations, as shown in fig. 21. The retrieval results reveal that the long-context retrieval ability of Multiscreen is not primarily driven by positional encoding. Replacing MiPE with RoPE severely degrades retrieval accuracy, even when selecting the best-performing RoPE scaling factor for each setting. In contrast, the NoPE variant retains much higher retrieval performance than the RoPE variant, especially with SWE. This suggests that the core screening mechanism, rather than positional extrapolation alone, is

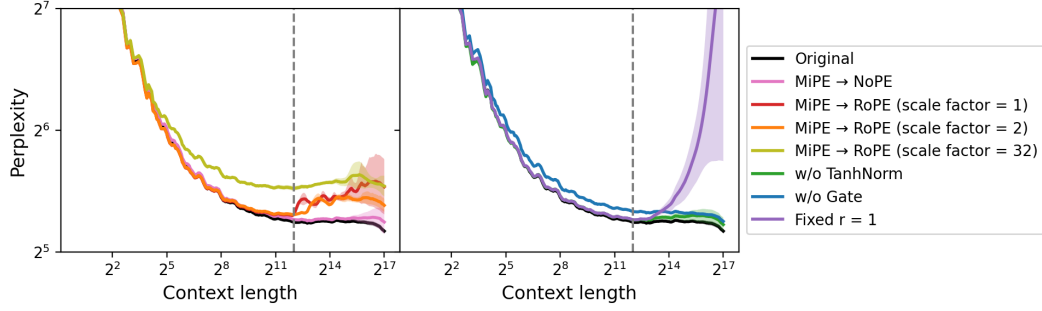


Figure 20: Long-context perplexity ablations for 28M Multiscreen variants. The left panel shows positional-encoding ablations, and the right panel shows TanhNorm, gate, and acceptance-width ablations. Curves show the mean over three independently trained models, and shaded regions indicate one standard deviation. The dashed vertical line marks the pretraining context length (2^{12}). For the RoPE ablation, representative RoPE scaling factors are shown. Fixing $r = 1$ causes sharp degradation at long contexts, while replacing MiPE with RoPE also worsens extrapolation. Replacing MiPE with NoPE, removing TanhNorm, and removing the gate have smaller effects on perplexity.

responsible for much of the long-context retrieval behavior, while MiPE further improves robustness under finite-window inference.

Removing TanhNorm substantially degrades ABCDigits retrieval, especially at longer contexts. This suggests that bounding the aggregation output norm helps make retrieval more reliable when relevance values are not normalized across keys. Removing the gate has little effect on ABCDigits retrieval, especially compared with its large effect on validation loss. This suggests that the gate mainly improves language-modeling performance rather than serving as the primary source of long-context retrieval ability. Fixing $r = 1$ also degrades retrieval, showing that learning the acceptance width is important for content-based screening. Although fixed $r = 1$ retains some retrieval ability, the learned acceptance width provides better selectivity and robustness.

Overall, these ablations support the view that screening is the central mechanism behind Multiscreen’s retrieval behavior. The strong NoPE results indicate that the gains are not simply due to positional extrapolation, while the severe RoPE degradation shows that inappropriate positional rotations can harm screening. At the same time, MiPE, TanhNorm, gating, and the learned acceptance width improve robustness in complementary ways. Across validation loss, long-context perplexity, and retrieval, the original architecture provides the best overall balance.

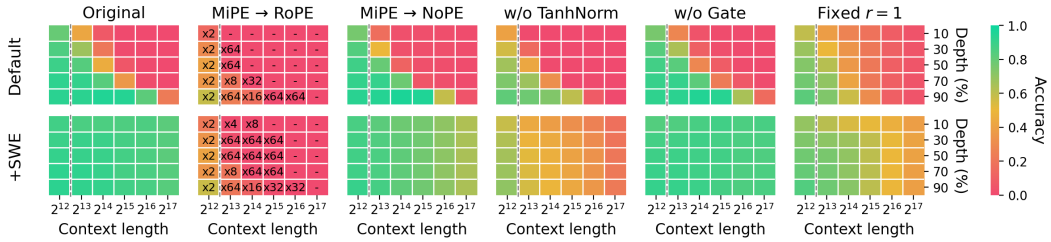


Figure 21: ABCDigits retrieval ablations for 28M Multiscreen variants. Top row: default finite-window inference. Bottom row: inference-time screening-window expansion (SWE), where learned screening windows exceeding the training context length are set to $w = \infty$. For the RoPE ablation, each cell reports accuracy under the best-performing RoPE scaling factor selected from multiple candidates, with the selected factor indicated in the cell. Replacing MiPE with RoPE severely degrades retrieval. NoPE performs substantially better than RoPE but remains weaker than the original model under finite-window inference. Removing TanhNorm and fixing $r = 1$ also reduce retrieval accuracy, while removing the gate has little effect in this retrieval evaluation. SWE improves long-context coverage for several ablations.

Q Limitations

This work focuses on evaluating Multiscreen as a language-model architecture for long-context modeling, retrieval, optimization stability, and inference efficiency. Several limitations remain.

First, our experiments are conducted at moderate model scales compared to modern frontier language models. Although we evaluate models up to 4B parameters for Multiscreen and 1.3B parameters for Transformer baselines, these models are not intended to be competitive general-purpose foundation models. In particular, we do not evaluate broad downstream knowledge or reasoning benchmarks such as MMLU [54] or GSM8K [55]. These benchmarks require substantial world knowledge, reasoning ability, and often benefit from larger-scale training or instruction tuning; our evaluation instead focuses on controlled architecture comparisons under matched token budgets and on isolating long-context retrieval behavior.

Second, most experiments use a fixed token budget. This enables controlled comparisons across architectures and model sizes, but it may undertrain larger models relative to smaller ones. For example, the 4B Multiscreen model deviates from the fitted scaling trend in Appendix E, which we attribute to undertraining at this scale. Training larger models with larger token budgets may yield different scaling behavior. In addition, the architectural ablations in Appendix P are trained for a shorter budget of 2^{36} tokens rather than the 2^{38} tokens used in the main experiments, and should therefore be interpreted as diagnostic comparisons rather than fully retuned training runs.

Third, our retrieval evaluations are intentionally controlled. ABCDigits removes natural-language semantics and instruction-following effects in order to isolate retrieval behavior, and passkey retrieval provides a complementary long-context retrieval test. However, these benchmarks do not cover all forms of real-world retrieval, such as document-level question answering, retrieval-augmented generation, multi-hop reasoning, or tasks requiring semantic understanding. Thus, the reported retrieval improvements should be interpreted as evidence for improved controlled retrieval behavior rather than as a complete evaluation of all long-context capabilities.

Fourth, not all experiments are replicated at every scale. For smaller models, several results are averaged over three independently trained models, but the largest models are evaluated using a single trained model due to computational cost. Some diagnostic experiments, such as the QKNorm Transformer baseline in Appendix I, are also run with fewer seeds than the main comparisons. We therefore emphasize qualitative trends that are consistent across model sizes and, where available, across random seeds, rather than making fine-grained claims about small numerical differences at the largest scales or in diagnostic ablations.

Fifth, the efficiency results depend on implementation and hardware. Transformer baselines use optimized scaled dot-product attention with the FlashAttention backend explicitly enabled, while Multiscreen uses a custom Triton implementation. Latency is measured on an RTX 4090 GPU with batch size 1 and no KV caching, using a full forward pass over the input sequence. Notably, FlashAttention is highly optimized for modern high-end GPU architectures (e.g., NVIDIA H100), whereas our Triton implementation is not specifically tuned for particular hardware. Different hardware, batch sizes, or kernel implementations may therefore change the absolute latency values and potentially the relative comparison, although the experiments in section 4.6 provide a controlled comparison under the stated setup. A more comprehensive study of accuracy–latency tradeoffs with hardware-specific optimization of screening kernels, for example on H100-class GPUs, is left for future work.

Finally, our ablation study is not exhaustive. Appendix P provides diagnostic ablations of MiPE, TanhNorm, gating, and the learned acceptance width, but we do not exhaustively explore all possible alternatives to Trim, Softmask, thresholding functions, gating mechanisms, or positional schemes. Some ablations also modify architectural capacity slightly; for example, the no-gate variant is adjusted in depth and number of heads to keep the parameter count close to the original model. Further parameter-matched ablations and larger-scale component studies are left for future work.

R Compute Resources and Experimental Cost

We provide details on the compute resources used for training and evaluation, along with approximate training and total compute.

Hardware. Training experiments were conducted on NVIDIA GH200 GPUs, except for the learning-rate sweep experiments, which were conducted on a mix of NVIDIA GH200 and H100 GPUs using between 4 and 16 GPUs depending on availability. The GPUs used for training and evaluation had at least 80GB of GPU memory.

Evaluation experiments were conducted on a mix of NVIDIA GH200 and H100 GPUs, depending on availability. Latency measurements were performed separately on an NVIDIA RTX 4090 GPU with 24GB memory.

Training cost. Table 6 summarizes the approximate training compute required for representative models across scales. Training cost is reported in GPU hours.

Table 6: Approximate training compute across model scales.

Model	# GPUs	Training cost (GPU hours)
8M Transformer	16	~85
45M Transformer	16	~240
353M Transformer	64	~1.4k
1.3B Transformer	64	~5.4k
4M Multiscreen	16	~250
28M Multiscreen	16	~630
286M Multiscreen	64	~2.0k
1.3B Multiscreen	64	~4.9k
4B Multiscreen	64	~13k

Evaluation cost. Evaluation experiments, including long-context perplexity, ABCDigits, passkey retrieval, and latency measurements, were conducted using a single GPU per run (GH200 or H100, with latency measurements on an RTX 4090). In total, these experiments required approximately 1k GPU hours, with a significant portion of the cost arising from repeated evaluations across multiple RoPE scaling factors for Transformer models.

Total compute. Reproducing all experiments reported in this paper (including the appendix) requires approximately 60k GPU hours. The total compute used in this project, including additional preliminary experiments, hyperparameter tuning, and failed runs, is approximately 200k GPU hours.

S Existing Assets and Licenses

We summarize the existing datasets, tokenizers, and software packages used in this work, together with their licenses and terms of use.

Datasets. We use SlimPajama [38] for pretraining. SlimPajama is a cleaned and deduplicated dataset derived from RedPajama [43], and is released under the Apache License 2.0 by Cerebras.³ RedPajama itself aggregates multiple data sources with heterogeneous licenses, and we rely on the SlimPajama release to ensure compliance with the underlying data sources.

For long-context perplexity evaluation, we use PG-19 [52]. The PG-19 repository is released under the Apache License 2.0⁴, and the benchmark consists of books extracted from Project Gutenberg that were published before 1919. Project Gutenberg states that its texts are generally unrestricted for use in the United States, while users outside the United States should verify the copyright status under their local laws.

Tokenization. We use the GPT-2 tokenizer [42]. The GPT-2 code and tokenizer assets are released by OpenAI under a modified MIT license.⁵

Software. Our implementation uses PyTorch [56] for neural-network training and inference. PyTorch is distributed under a BSD-style license.⁶ For Transformer baselines, we use PyTorch scaled dot-product attention with the FlashAttention backend explicitly enabled; FlashAttention [40] is released under the BSD-3-Clause license.⁷ For Multiscreen kernels, we use Triton [41], which is released under the MIT license.⁸ All assets are used in accordance with their respective licenses and terms of use.

³<https://www.cerebras.ai/blog/slimpajama-a-627b-token-cleaned-and-deduplicated-version-of-redpajama>

⁴<https://github.com/google-deepmind/pg19>

⁵<https://github.com/openai/gpt-2>

⁶<https://github.com/pytorch/pytorch>

⁷<https://github.com/Dao-AILab/flash-attention>

⁸<https://github.com/triton-lang/triton>

T Broader Impacts

This work studies architectural improvements for language models, particularly in long-context understanding and retrieval.

On the positive side, improved long-context modeling can benefit applications such as document analysis, information retrieval, and scientific understanding, where reasoning over extended context is important.

On the negative side, improved language modeling capabilities may also increase the potential for misuse, such as generating misleading or harmful content at scale. As with other advances in language models, responsible deployment and monitoring are important to mitigate such risks.